AI-LangX Architecture Deep Dive

LangChain + LangGraph + LangSmith Implementation Guide

Date: February 9, 2026
Purpose: Educational reference and demo guide
Audience: Developers learning LangChain/LangGraph/LangSmith
Implementation: ai-langx/ (Production-grade framework-based AI orchestrator)

Table of Contents

1. Overview
2. Functional Flow
3. Code Flow Deep Dive
4. LangChain Concepts
5. LangGraph Workflow Execution
6. LangSmith Observability
7. End-to-End Examples
8. Custom vs Framework Comparison
9. Demo Script

---

1. Overview

1.1 What is ai-langx?

ai-langx/ is a production-grade AI orchestrator that demonstrates enterprise patterns using LangChain, LangGraph, and LangSmith. It serves as both:

- **Production System**: Fully functional AI layer for expense tracking
- **Learning Resource**: Educational reference for framework adoption
- **Comparison Baseline**: Side-by-side with custom implementation (ai/)

1.2 Architecture at a Glance

```
┌──────────────────────────────────────────────────────────────────┐
│                    FRONTEND (Angular)                            │
│                POST /ai/chat | POST /ai/upload                   │
└──────────────────────────────────────────────────────────────────┘
                              │
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│               AI-LANGX EXPRESS SERVER (Port 3002)                │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │          INTENT ROUTER GRAPH (LangGraph)                 │   │
```

```
│  │  ┌──────────────┐                                          │    │
│  │  │ Classify     │──┐                                       │    │
│  │  │ Intent (LLM) │  │                                       │    │
│  │  └──────────────┘  │                                       │    │
│  │                    ├──> expense_operation ──> Agent        │    │
│  │                    ├──> rag_question ──> RAG Chain         │    │
│  │                    ├──> reconciliation ──> Recon Graph     │    │
│  │                    └──> general_chat ──> Simple Response   │    │
│  └──────────────────────────────────────────────────────────┘    │
│                                                                    │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │          LANGCHAIN COMPONENTS                            │    │
│  │  • AgentExecutor (tool-calling)                          │    │
│  │  • StructuredTools (5 expense tools)                     │    │
│  │  • RetrievalQAChain (PDF Q&A)                            │    │
│  │  • MemoryVectorStore (embeddings)                        │    │
│  │  • RecursiveCharacterTextSplitter (chunking)             │    │
│  └──────────────────────────────────────────────────────────┘    │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼
┌────────────────────────────────────────────────────────────────────┐
│                   LANGSMITH PLATFORM                               │
│         (Automatic tracing, debugging, analytics)                  │
└────────────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌────────────────────────────────────────────────────────────────────┐
│                    BACKEND APIs (Port 3000)                        │
│          /expenses | /categories | /auth (unchanged)               │
└────────────────────────────────────────────────────────────────────┘
```

1.3 Key Technologies

Technology	Purpose	Benefits
LangChain	AI orchestration framework	- Standard patterns - Reusable components - 500+ integrations
LangGraph	Stateful workflow engine	- Visual workflows - Conditional routing - State persistence
LangSmith	Observability platform	- Automatic tracing - Visual debugging - Cost tracking
Zod	Runtime validation	- Type safety - Error messages - Auto schema conversion

Technology	Purpose	Benefits
OpenAI	LLM provider	- Function calling - Embeddings - Chat completions

2. Functional Flow

2.1 High-Level Flow

```
User Message
    │
    ▼
EXPRESS MIDDLEWARE
├─ CORS validation
├─ Rate limiting
├─ JWT authentication
├─ Body parsing
└─ Error handling
    │
    ▼
INTENT ROUTER GRAPH
├─ Node: Classify Intent (LLM)
│   ├─ Input: User message + history
│   ├─ LLM: GPT-4o-mini (temp=0)
│   └─ Output: intent + confidence + entities
│
└─ Conditional Edges (Router)
    │
    ├─> expense_operation ──> AGENT EXECUTOR
    │                          ├─ Tool selection by LLM
    │                          ├─ Tool execution
    │                          └─ Response generation
    │
    ├─> rag_question ──> RAG QA CHAIN
    │                    ├─ Retrieve docs (vector search)
    │                    ├─ Format context
    │                    └─ Generate answer
    │
    ├─> reconciliation ──> RECONCILIATION GRAPH
    │                      ├─ Fetch app expenses
    │                      ├─ Compare with PDF
    │                      ├─ Generate report
    │                      └─ Optional sync
    │
    └─> general_chat ──> SIMPLE RESPONSE
                         └─ Help/greeting message
```

2.2 Request/Response Contract

Same as custom implementation - Frontend doesn't change!

Request (`POST /ai/chat`):

```
{
  "message": "Add 500 for lunch today",
  "history": [
    { "role": "user", "content": "Previous message..." },
    { "role": "assistant", "content": "Previous response..." }
  ]
}
```

Response:

```
{
  "reply": "✅ Successfully added ₹500 for Food on 2026-02-09. Total today:
₹500."
}
```

2.3 Component Responsibilities

```
ai-langx/
│
├── server.js                    # Express setup + LangSmith init
│
├── src/routes/
│   ├── chat.js                  # POST /ai/chat (main endpoint)
│   ├── upload.js                # POST /ai/upload (PDF ingestion)
│   └── reconcile.js             # POST /ai/reconcile (direct recon)
│
├── src/graphs/                  # LangGraph workflows
│   ├── state.js                 # Zod state schemas
│   ├── intent-router.graph.js   # Intent classification + routing
│   └── reconciliation.graph.js  # Multi-step reconciliation
│
├── src/agents/
│   └── expense.agent.js         # AgentExecutor for tool-calling
│
├── src/tools/                   # 5 StructuredTools
│   ├── createExpense.tool.js    # Create expense (Zod + backend)
│   ├── listExpenses.tool.js     # List/filter expenses
│   ├── modifyExpense.tool.js    # Update expense
│   ├── deleteExpense.tool.js    # Delete expense
│   └── clearExpenses.tool.js    # Clear all expenses
│
├── src/rag/                     # RAG pipeline
│   ├── loaders/pdf.loader.js    # PDF text extraction
│   ├── splitters/               # Text chunking
```

```
│   ├── embeddings/              # OpenAI embeddings
│   ├── vectorstore/             # MemoryVectorStore + persistence
│   ├── retrievers/              # User-filtered retrieval
│   └── chains/qa.chain.js       # RetrievalQAChain
│
├── src/handlers/
│   └── rag.handler.js           # RAG Q&A handler
│
├── src/prompts/
│   └── system.prompt.js         # ChatPromptTemplate
│
├── src/config/
│   ├── langsmith.config.js      # LangSmith setup
│   └── llm.config.js            # OpenAI config
│
└── src/utils/
    ├── backendClient.js         # Axios client
    ├── dateNormalizer.js        # Date parsing
    └── helpers.js               # Utility functions
```

---

3. Code Flow Deep Dive

3.1 Request Entry Point

File: `src/routes/chat.js`

```
router.post('/chat', authMiddleware, async (req, res, next) => {
  const traceId = generateTraceId();
  const userId = req.user?.userId;

  // 1. Validate request
  const { message, history } = req.body;

  // 2. Invoke intent router graph
  const result = await executeIntentRouter({
    userMessage: message,
    userId,
    authToken: req.token,
    conversationHistory: history || []
  });

  // 3. Return response
  return res.json({ reply: result.result });
});
```

Flow:

1. Express middleware validates JWT → extracts `userId`
2. Request body validation (Zod schemas optionally)

3. Generate `traceId` for this request
4. Invoke LangGraph workflow (`executeIntentRouter`)
5. Return response in same format as custom implementation

3.2 Intent Classification (LangGraph Node)

File: `src/graphs/intent-router.graph.js`

Node Function:

```
const classifyIntent = async (state) => {
  // 1. Create LLM for classification
  const llm = new ChatOpenAI({
    modelName: "gpt-4o-mini",
    temperature: 0  // Deterministic for classification
  });

  // 2. Build conversation context
  let conversationContext = '';
  if (state.conversationHistory.length > 0) {
    const recentHistory = state.conversationHistory.slice(-6);
    recentHistory.forEach(msg => {
      conversationContext += `${msg.role}: ${msg.content}\n`;
    });
  }

  // 3. Create classification prompt
  const classificationPrompt = PromptTemplate.fromTemplate(`
You are an intent classifier for an expense tracking application.

Classify into ONE of these intents:
1. expense_operation - Create, list, modify, delete expenses
2. rag_question - Questions about uploaded PDFs
3. reconciliation - Sync/reconcile expenses
4. general_chat - Greetings, help

Recent conversation:
{context}

User message: "{message}"

Return JSON:
{{"intent": "expense_operation", "confidence": 0.95}}
`);

  // 4. Invoke LLM
  const prompt = await classificationPrompt.format({
    context: conversationContext,
    message: state.userMessage
  });

  const response = await llm.invoke(prompt);
```

```
    const classification = JSON.parse(response.content);

    // 5. Return state update
    return {
      intent: classification.intent,
      confidence: classification.confidence,
      reasoning: classification.reasoning || ''
    };
  };
```

LangGraph Compilation:

```
const workflow = new StateGraph(IntentRouterStateSchema)
  .addNode("classifyIntent", classifyIntent)
  .addNode("handleExpenseOperation", handleExpenseOperation)
  .addNode("handleRAGQuestion", handleRAGQuestion)
  .addNode("handleReconciliation", handleReconciliation)
  .addNode("handleGeneralChat", handleGeneralChat)

  .addEdge(START, "classifyIntent")

  // Conditional routing based on intent
  .addConditionalEdges(
    "classifyIntent",
    (state) => state.intent,
    {
      "expense_operation": "handleExpenseOperation",
      "rag_question": "handleRAGQuestion",
      "reconciliation": "handleReconciliation",
      "general_chat": "handleGeneralChat"
    }
  )

  .addEdge("handleExpenseOperation", END)
  .addEdge("handleRAGQuestion", END)
  .addEdge("handleReconciliation", END)
  .addEdge("handleGeneralChat", END);

export const intentRouterGraph = workflow.compile();
```

Execution:

```
const result = await intentRouterGraph.invoke({
  userMessage: "Add 500 for lunch",
  userId: 123,
  authToken: "jwt...",
  conversationHistory: []
});
```

```
// Result: { intent: "expense_operation", result: "☑ Added..." }
```

3.3 Tool-Calling Flow (AgentExecutor)

File: `src/agents/expense.agent.js`

Agent Creation:

```
export const createExpenseAgent = async (authToken, context = {}) => {
  // 1. Create LLM
  const llm = createLLM({
    temperature: 0.7,  // Natural tool usage
    tags: getTraceTags('transactional', context.userId),
    metadata: getTraceMetadata(context.traceId, context.userId)
  });

  // 2. Create tools with context
  const tools = createToolsWithContext(authToken, context);

  // 3. Create prompt
  const prompt = ChatPromptTemplate.fromMessages([
    ["system", getSystemPromptText()],
    ["placeholder", "{chat_history}"],
    ["human", "{input}"],
    ["placeholder", "{agent_scratchpad}"]  // For tool execution
  ]);

  // 4. Create agent (binds LLM + tools)
  const agent = await createOpenAIToolsAgent({
    llm,
    tools,
    prompt
  });

  // 5. Create executor (manages loop)
  return new AgentExecutor({
    agent,
    tools,
    maxIterations: 5,
    returnIntermediateSteps: true,
    handleParsingErrors: true,
    verbose: true
  });
};
```

Execution:

```
const agent = await createExpenseAgent(authToken, { userId, traceId });

const result = await agent.invoke({
  input: "Add 500 for lunch today",
  chat_history: []
});

// AgentExecutor automatically:
// 1. Passes tool schemas to LLM
// 2. LLM decides to call create_expense tool
// 3. Executor validates args (Zod)
// 4. Executor executes tool._call()
// 5. Executor adds result to conversation
// 6. LLM sees result and generates final response
// 7. Executor returns when LLM stops calling tools
```

3.4 Tool Execution (StructuredTool)

File: `src/tools/createExpense.tool.js`

Tool Definition:

```
export class CreateExpenseTool extends StructuredTool {
  name = "create_expense";

  description = "Add a new expense to the database. Use when user wants to
add/create/record an expense.";

  // Zod schema (auto-converted to OpenAI function schema)
  schema = z.object({
    amount: z.number().positive().describe("Amount in numbers only"),
    category: z.string().min(1).describe("Category name from user's message"),
    description: z.string().default("").describe("Optional description"),
    date: z.string().optional().describe("Date (today, yesterday, or YYYY-MM-DD)")
  });

  // Context injection
  constructor(authToken, context) {
    super();
    this.authToken = authToken;
    this.userId = context.userId;
    this.traceId = context.traceId;
  }

  // Implementation (args already validated by Zod)
  async _call(args) {
    // 1. Normalize inputs
    const normalizedCategory = await normalizeCategory(args.category,
this.authToken);
    const normalizedDate = normalizeDateToISO(args.date || 'today');
```

```
      // 2. Validate business rules
      const validatedAmount = validateAmount(args.amount);

      // 3. Call backend API
      const backendClient = createBackendClient(this.authToken);
      const response = await backendClient.post('/expenses', {
        amount: validatedAmount,
        category_id: normalizedCategory.id,
        description: args.description || `Expense for ${normalizedCategory.name}`,
        date: normalizedDate
      });

      // 4. Return formatted result
      if (response.data.success) {
        return JSON.stringify({
          success: true,
          message: `Expense of ₹${validatedAmount} for ${normalizedCategory.name}
added successfully on ${normalizedDate}.`,
          expense: response.data.expense
        });
      } else {
        return JSON.stringify({
          success: false,
          error: response.data.error || 'Failed to create expense'
        });
      }
    }
  }
```

Tool Registration:

```
// src/tools/index.js
export const createToolsWithContext = (authToken, context) => {
  return [
    new CreateExpenseTool(authToken, context),
    new ListExpensesTool(authToken, context),
    new ModifyExpenseTool(authToken, context),
    new DeleteExpenseTool(authToken, context),
    new ClearExpensesTool(authToken, context)
  ];
};
```

3.5 RAG Pipeline Flow

File: `src/rag/chains/qa.chain.js`

QA Chain Creation:

```
export const createQAChain = async (userId, options = {}) => {
  // 1. Get vector store (singleton)
  const vectorStore = await getVectorStore();

  // 2. Create retriever with userId filter
  const filterFunc = (doc) => {
    return doc.metadata && doc.metadata.userId === userId;
  };

  const retriever = vectorStore.asRetriever({
    k: 5,  // Top 5 chunks
    filter: filterFunc,
    searchType: "similarity"
  });

  // 3. Create LLM
  const llm = new ChatOpenAI({
    modelName: "gpt-4o-mini",
    temperature: 0  // Factual answers
  });

  // 4. Create prompt template
  const qaPrompt = PromptTemplate.fromTemplate(`
You are an AI assistant answering questions about uploaded PDF documents.

Context:
{context}

Question: {question}

Instructions:
- Answer based ONLY on the provided context
- If answer not in context, say "I don't have enough information..."

Answer:`);

  // 5. Create chain (automatic RAG)
  const chain = RetrievalQAChain.fromLLM(llm, retriever, {
    prompt: qaPrompt,
    returnSourceDocuments: true,
    verbose: true
  });

  return chain;
};
```

Usage:

```
export const answerQuestion = async (question, userId) => {
  const chain = await createQAChain(userId);
```

```
    // Chain automatically:
    // 1. Embeds question using OpenAIEmbeddings
    // 2. Searches vectorStore with cosine similarity
    // 3. Retrieves top-k chunks (filtered by userId)
    // 4. Formats chunks into context string
    // 5. Prompts LLM with context + question
    // 6. Returns answer + source documents

    const result = await chain.call({ query: question });

    return {
      answer: result.text,
      sources: result.sourceDocuments.map(doc => ({
        filename: doc.metadata.filename,
        snippet: doc.pageContent.substring(0, 200)
      }))
    };
  };
```

3.6 Reconciliation Graph Flow

File: `src/graphs/reconciliation.graph.js`

Multi-Stage Workflow:

```
const workflow = new StateGraph(ReconciliationStateSchema)
  // Node definitions
  .addNode("initializeReconciliation", initializeReconciliation)
  .addNode("fetchAppExpenses", fetchAppExpenses)
  .addNode("fetchDocumentDetails", fetchDocumentDetails)
  .addNode("comparePDFVsApp", comparePDFVsApp)
  .addNode("generateReport", generateReport)
  .addNode("respondToUser", respondToUser)

  // Flow definition
  .addEdge(START, "initializeReconciliation")
  .addEdge("initializeReconciliation", "fetchAppExpenses")
  .addEdge("initializeReconciliation", "fetchDocumentDetails")  // Parallel!

  // Wait for both before comparing
  .addEdge("fetchAppExpenses", "comparePDFVsApp")
  .addEdge("fetchDocumentDetails", "comparePDFVsApp")

  .addEdge("comparePDFVsApp", "generateReport")
  .addEdge("generateReport", "respondToUser")
  .addEdge("respondToUser", END);

export const reconciliationGraph = workflow.compile();
```

Stage-by-Stage:

Stage 1: Initialize

```
const initializeReconciliation = async (state) => {
  if (!state.documentExpenses || state.documentExpenses.length === 0) {
    return { error: 'No document expense data provided' };
  }

  return {
    stage: 'fetch_app_expenses',
    traceId: `recon-${Date.now()}`,
    timestamp: new Date().toISOString()
  };
};
```

Stage 2 & 3: Parallel Fetch (LangGraph waits for both)

```
const fetchAppExpenses = async (state) => {
  const backendClient = createBackendClient(state.authToken);
  const response = await backendClient.get('/expenses');

  return {
    appExpenses: response.data.expenses,
    stage: 'compare'
  };
};

const fetchDocumentDetails = async (state) => {
  const pdfDocs = await getUserDocuments(state.userId);

  return {
    documentDetails: pdfDocs,
    stage: 'compare'
  };
};
```

Stage 4: Compare (Deterministic algorithm)

```
const comparePDFVsApp = async (state) => {
  const matches = [];
  const discrepancies = [];

  for (const docExpense of state.documentExpenses) {
    let bestMatch = null;
    let bestScore = 0;

    for (const appExpense of state.appExpenses) {
      const score = calculateMatchScore(docExpense, appExpense);
      if (score > bestScore) {
```

```
          bestScore = score;
          bestMatch = appExpense;
        }
      }

      if (bestScore >= 0.9) {
        matches.push({ documentExpense, appExpense: bestMatch, matchScore: bestScore
});
      } else {
        discrepancies.push({ type: 'missing_in_app', documentExpense });
      }
    }

    return { matches, discrepancies };
};
```

Stage 5: Generate Report

```
const generateReport = async (state) => {
  const report = {
    totalMatched: state.matches.length,
    totalDiscrepancies: state.discrepancies.length,
    matches: state.matches,
    discrepancies: state.discrepancies,
    generatedAt: new Date().toISOString()
  };

  return { report };
};
```

Stage 6: Respond

```
const respondToUser = async (state) => {
  const summary = `
Reconciliation Complete:
- Matched: ${state.matches.length}
- Discrepancies: ${state.discrepancies.length}
- Missing in app: ${state.discrepancies.filter(d => d.type ===
'missing_in_app').length}
  `;

  return { result: summary };
};
```

---

4. LangChain Concepts

4.1 Core Components Used

1. Models (LLMs)

```
import { ChatOpenAI } from "@langchain/openai";

const llm = new ChatOpenAI({
  modelName: "gpt-4o-mini",
  temperature: 0.7,
  openAIApiKey: process.env.OPENAI_API_KEY,

  // LangSmith tracing
  tags: ['expense-tracker', 'transactional', 'user:123'],
  metadata: { traceId: 'abc123', userId: 123 }
});

const response = await llm.invoke("What is 2+2?");
```

Benefits:

- Consistent interface across providers (OpenAI, Anthropic, etc.)
- Automatic retry logic
- Token usage tracking
- LangSmith integration

2. Prompts

```
import { PromptTemplate, ChatPromptTemplate } from "@langchain/core/prompts";

// Simple template
const prompt = PromptTemplate.fromTemplate(`
You are a {role}.
User: {input}
Answer:
`);

const formatted = await prompt.format({
  role: "expense assistant",
  input: "Add 500 for lunch"
});

// Chat template with messages
const chatPrompt = ChatPromptTemplate.fromMessages([
  ["system", "You are an expense assistant."],
  ["human", "{input}"],
  ["placeholder", "{agent_scratchpad}"]  // For agents
]);
```

Benefits:

- Template reuse
- Variable validation
- Few-shot examples
- LangSmith prompt versioning

3. Tools (StructuredTool)

```
import { StructuredTool } from "@langchain/core/tools";
import { z } from "zod";

class MyTool extends StructuredTool {
  name = "my_tool";
  description = "What this tool does";
  schema = z.object({
    arg1: z.string().describe("First argument"),
    arg2: z.number().describe("Second argument")
  });

  async _call(args) {
    // Implementation
    return "Result";
  }
}

// Usage:
const tool = new MyTool();
const result = await tool._call({ arg1: "hello", arg2: 42 });
```

Benefits:

- Automatic OpenAI function schema conversion
- Zod validation
- Error handling
- LangSmith tracing

4. Agents (AgentExecutor)

```
import { AgentExecutor, createOpenAIToolsAgent } from "langchain/agents";

const agent = await createOpenAIToolsAgent({
  llm,
  tools: [tool1, tool2, tool3],
  prompt: chatPrompt
});

const executor = new AgentExecutor({
  agent,
  tools,
  maxIterations: 5,
```

```
  returnIntermediateSteps: true
});

const result = await executor.invoke({
  input: "User message",
  chat_history: []
});
```

Automatic Behavior:

- Tool schema passing to LLM
- Tool call parsing
- Tool execution
- Loop until completion
- Error recovery

5. Chains (RetrievalQAChain)

```
import { RetrievalQAChain } from "langchain/chains";

const chain = RetrievalQAChain.fromLLM(llm, retriever, {
  prompt: qaPrompt,
  returnSourceDocuments: true
});

const result = await chain.call({ query: "What's in my PDFs?" });
// Returns: { text: "Answer...", sourceDocuments: [...] }
```

Automatic Behavior:

- Query → Retrieve → Format → Generate
- Source attribution
- Context management

6. Document Loaders

```
import { PDFLoader } from "langchain/document_loaders/fs/pdf";

const loader = new PDFLoader("./document.pdf");
const docs = await loader.load();
```

Benefits:

- Consistent document format
- Metadata handling
- Multiple loaders (PDF, CSV, JSON, etc.)

7. Text Splitters

```
import { RecursiveCharacterTextSplitter } from "langchain/text_splitter";

const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 1500,
  chunkOverlap: 200,
  separators: ["\n\n", "\n", ".", " "]
});

const chunks = await splitter.createDocuments([text], [metadata]);
```

Benefits:

- Semantic chunking
- Overlap for context preservation
- Metadata propagation

8. Embeddings

```
import { OpenAIEmbeddings } from "@langchain/openai";

const embeddings = new OpenAIEmbeddings({
  modelName: "text-embedding-3-small",
  dimensions: 1536
});

const vector = await embeddings.embedQuery("Search query");
const vectors = await embeddings.embedDocuments(["Doc 1", "Doc 2"]);
```

Benefits:

- Consistent interface
- Batch processing
- Caching support

9. Vector Stores

```
import { MemoryVectorStore } from "langchain/vectorstores/memory";

const vectorStore = new MemoryVectorStore(embeddings);

// Add documents (automatic embedding)
await vectorStore.addDocuments(docs);

// Search
const results = await vectorStore.similaritySearch("query", 5);
```

```
const resultsWithScores = await vectorStore.similaritySearchWithScore("query", 5);

// Create retriever
const retriever = vectorStore.asRetriever({
  k: 5,
  filter: (doc) => doc.metadata.userId === 123
});
```

Benefits:

- Standard interface (easy to swap stores)
- Automatic embedding
- Retriever abstraction

4.2 Composition Patterns

Pattern 1: LLM + Prompt

```
const llm = new ChatOpenAI({ modelName: "gpt-4o-mini" });
const prompt = PromptTemplate.fromTemplate("You are {role}. {input}");

const chain = prompt.pipe(llm);
const result = await chain.invoke({ role: "assistant", input: "Hello" });
```

Pattern 2: Tool-Calling Agent

```
const tools = [createExpenseTool, listExpenseTool];
const agent = await createOpenAIToolsAgent({ llm, tools, prompt });
const executor = new AgentExecutor({ agent, tools });
```

Pattern 3: RAG Pipeline

```
const vectorStore = new MemoryVectorStore(embeddings);
const retriever = vectorStore.asRetriever();
const chain = RetrievalQAChain.fromLLM(llm, retriever);
```

---

5. LangGraph Workflow Execution

5.1 Core Concepts

StateGraph

- Stateful workflow engine

- Nodes: Functions that transform state
- Edges: Connections between nodes
- State: Travels through the graph

State Schema (Zod)

```
const MyStateSchema = z.object({
  input: z.string(),
  output: z.string().optional(),
  error: z.string().optional()
});
```

Node Function

```
const myNode = async (state) => {
  // Read from state
  const { input } = state;

  // Do work
  const result = await doSomething(input);

  // Return state updates (not full state)
  return { output: result };
};
```

Graph Compilation

```
const workflow = new StateGraph(MyStateSchema)
  .addNode("node1", node1Function)
  .addNode("node2", node2Function)
  .addEdge(START, "node1")
  .addEdge("node1", "node2")
  .addEdge("node2", END);

const app = workflow.compile();
```

Execution

```
const result = await app.invoke({ input: "Hello" });
// State flows through: START → node1 → node2 → END
```

5.2 Edge Types

Simple Edge

```
.addEdge("nodeA", "nodeB")  // Always go from A to B
```

Conditional Edge

```
.addConditionalEdges(
  "nodeA",
  (state) => state.intent,  // Routing function
  {
    "option1": "nodeB",
    "option2": "nodeC",
    "option3": END
  }
)
```

START and END

```
import { START, END } from "@langchain/langgraph";

.addEdge(START, "firstNode")  // Entry point
.addEdge("lastNode", END)     // Exit point
```

5.3 Parallel Execution

```
const workflow = new StateGraph(State)
  .addNode("node1", node1)
  .addNode("node2", node2)
  .addNode("node3", node3)

  // Both start from START (parallel)
  .addEdge(START, "node1")
  .addEdge(START, "node2")

  // Node3 waits for both
  .addEdge("node1", "node3")
  .addEdge("node2", "node3")

  .addEdge("node3", END);
```

LangGraph automatically:

- Executes node1 and node2 in parallel

- Waits for both to complete
- Merges their state updates
- Proceeds to node3

5.4 State Reducers

Default Behavior: Last write wins

```
// Node1 returns: { output: "A" }
// Node2 returns: { output: "B" }
// Result: { output: "B" }  ← Last one wins
```

Custom Reducer:

```
const workflow = new StateGraph(State, {
  reducer: (existingState, updates) => {
    // Custom merge logic
    return {
      ...existingState,
      ...updates,
      // Append to arrays instead of replacing
      items: [...existingState.items, ...updates.items]
    };
  }
});
```

5.5 Error Handling

```
const myNode = async (state) => {
  try {
    const result = await riskyOperation();
    return { output: result };
  } catch (error) {
    console.error('Node error:', error);
    return { error: error.message };
  }
};

// Conditional routing on error
.addConditionalEdges(
  "myNode",
  (state) => state.error ? "error" : "success",
  {
    "success": "nextNode",
    "error": "errorHandler"
  }
)
```

5.6 Intent Router Graph (Real Example)

Visual Flow:

```
                START
                  |
                  ▼
          ┌──────────────┐
          │  CLASSIFY    │  (LLM classifies intent)
          │   INTENT     │
          └──────┬───────┘
                 |
        ┌────────┴──────────┬──────────┬──────────┐
        │                   │          │          │
        ▼                   ▼          ▼          ▼
   ┌─────────┐        ┌──────────┐  ┌──────┐  ┌─────────┐
   │ Expense │        │ RAG Q&A  │  │Recon │  │ General │
   │Operation│        │          │  │      │  │  Chat   │
   └────┬────┘        └────┬─────┘  └──┬───┘  └────┬────┘
        │                  │           │           │
        └──────────────────┴───────────┴───────────┘
                           |
                           ▼
                          END
```

Code Implementation:

```
const workflow = new StateGraph(IntentRouterStateSchema)
  // Add nodes
  .addNode("classifyIntent", classifyIntentNode)
  .addNode("handleExpenseOperation", handleExpenseOperationNode)
  .addNode("handleRAGQuestion", handleRAGQuestionNode)
  .addNode("handleReconciliation", handleReconciliationNode)
  .addNode("handleGeneralChat", handleGeneralChatNode)

  // Start with classification
  .addEdge(START, "classifyIntent")

  // Route based on classified intent
  .addConditionalEdges(
    "classifyIntent",
    (state) => state.intent,
    {
      "expense_operation": "handleExpenseOperation",
      "rag_question": "handleRAGQuestion",
      "reconciliation": "handleReconciliation",
      "general_chat": "handleGeneralChat"
    }
  )
```

```
  // All handlers end the workflow
  .addEdge("handleExpenseOperation", END)
  .addEdge("handleRAGQuestion", END)
  .addEdge("handleReconciliation", END)
  .addEdge("handleGeneralChat", END);

export const intentRouterGraph = workflow.compile();
```

Execution Trace:

```
const result = await intentRouterGraph.invoke({
  userMessage: "Add 500 for lunch",
  userId: 123,
  authToken: "jwt...",
  conversationHistory: []
});

// LangSmith trace shows:
// 1. classifyIntent node: 1.2s
//    ├─ LLM call: 1.1s (GPT-4o-mini)
//    └─ Output: { intent: "expense_operation", confidence: 0.95 }
// 2. handleExpenseOperation node: 2.3s
//    ├─ AgentExecutor: 2.3s
//    │  ├─ LLM call #1: Tool selection (0.8s)
//    │  ├─ Tool: create_expense (1.2s)
//    │  │  └─ Backend API call (0.9s)
//    │  └─ LLM call #2: Generate response (0.3s)
//    └─ Output: { result: "✅ Added ₹500..." }
// Total: 3.5s
```

5.7 Reconciliation Graph (Real Example)

Visual Flow:

```
        START
          │
          ▼
     ┌─────────┐
     │  INIT   │
     └────┬────┘
          │
     ┌────┴────┐
     │         │
     ▼         ▼
┌─────────┐ ┌─────────┐
│ FETCH   │ │ FETCH   │  (Parallel!)
│  APP    │ │  PDF    │
└────┬────┘ └────┬────┘
```

```
        └──────────┘
             │
             ▼
        ┌─────────┐
        │ COMPARE │   (Deterministic)
        └─────────┘
             │
             ▼
        ┌────────┐
        │ REPORT │
        └────────┘
             │
             ▼
        ┌────────┐
        │ RESPOND │
        └────────┘
             │
             ▼
           END
```

Key Features:

- **Parallel Fetch**: Reduces latency by ~40%
- **Deterministic Compare**: No LLM hallucinations
- **State Accumulation**: Build report through stages
- **Error Recovery**: Retry logic in fetch nodes

---

6. LangSmith Observability

6.1 Setup

Environment Variables:

```
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=ls_abc123...
LANGCHAIN_PROJECT=expense-tracker-ai-langx
LANGCHAIN_ENDPOINT=https://api.smith.langchain.com
```

Initialization (`server.js`):

```
import { initializeLangSmith } from './src/config/langsmith.config.js';

// Initialize early in startup
initializeLangSmith();

console.log('[LangSmith] Tracing enabled:', process.env.LANGCHAIN_TRACING_V2);
```

Configuration (`src/config/langsmith.config.js`):

```
export const LANGSMITH_CONFIG = {
  ENABLED: process.env.LANGCHAIN_TRACING_V2 === 'true',
  API_KEY: process.env.LANGCHAIN_API_KEY,
  PROJECT: process.env.LANGCHAIN_PROJECT || 'expense-tracker-ai-langx'
};

export const getTraceTags = (intent, userId) => {
  return [
    'expense-tracker',
    intent,
    `user:${userId}`,
    `env:${process.env.NODE_ENV || 'development'}`
  ];
};

export const getTraceMetadata = (traceId, userId, additionalMeta = {}) => {
  return {
    traceId,
    userId,
    timestamp: new Date().toISOString(),
    environment: process.env.NODE_ENV,
    ...additionalMeta
  };
};
```

6.2 Automatic Tracing

What Gets Traced Automatically (Zero Code):

- ☑ Every LLM call (input, output, tokens, latency, cost)
- ☑ Every tool execution (name, args, result, duration)
- ☑ Every chain step (retrieval, prompt, generation)
- ☑ Every graph node (state before/after, transitions)
- ☑ Errors with full context
- ☑ Token usage aggregation
- ☑ Cost calculation

Example LLM Call with Tracing:

```
const llm = new ChatOpenAI({
  modelName: "gpt-4o-mini",
  temperature: 0.7,

  // Add tags for filtering
  tags: getTraceTags('transactional', userId),

  // Add metadata for context
  metadata: getTraceMetadata(traceId, userId, {
    sessionId: 'session123',
    feature: 'expense_creation'
```

```
  })
});

const response = await llm.invoke("Add 500 for lunch");

// LangSmith automatically captures:
// - Input: "Add 500 for lunch"
// - Output: Tool calls or text response
// - Tokens: { prompt: 45, completion: 23, total: 68 }
// - Latency: 1234ms
// - Cost: $0.000034
// - Tags: ['expense-tracker', 'transactional', 'user:123', 'env:development']
// - Metadata: { traceId, userId, sessionId, feature, timestamp }
```

6.3 LangSmith Dashboard

Trace Explorer:

```
┌─────────────────────────────────────────────────────────────┐
│  Trace: expense_creation_2026-02-09_abc123                  │
│  Duration: 3.5s | Cost: $0.00012 | Status: ☑ Success        │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│  ├─ IntentRouterGraph                                       │
│  │  ├─ Node: classifyIntent (1.2s)                          │
│  │  │  └─ ChatOpenAI (gpt-4o-mini)                          │
│  │  │     ├─ Input: "Add 500 for lunch"                     │
│  │  │     ├─ Output: {intent: "expense_operation"}          │
│  │  │     ├─ Tokens: 234 (prompt: 180, completion: 54)      │
│  │  │     └─ Cost: $0.000023                                │
│  │  │                                                       │
│  │  ├─ Node: handleExpenseOperation (2.3s)                  │
│  │  │  └─ AgentExecutor                                     │
│  │  │     ├─ LLM Call #1: Tool Selection (0.8s)             │
│  │  │     │  ├─ Input: System prompt + "Add 500..."         │
│  │  │     │  ├─ Output: tool_calls: [create_expense(...)]   │
│  │  │     │  └─ Tokens: 189                                 │
│  │  │     │                                                 │
│  │  │     ├─ Tool: create_expense (1.2s)                    │
│  │  │     │  ├─ Input: {amount: 500, category: "food"...}   │
│  │  │     │  ├─ Backend API Call (0.9s)                     │
│  │  │     │  └─ Output: {success: true, expense: {...}}     │
│  │  │     │                                                 │
│  │  │     └─ LLM Call #2: Generate Response (0.3s)          │
│  │  │        ├─ Input: Previous messages + tool result      │
│  │  │        ├─ Output: "☑ Successfully added ₹500..."      │
│  │  │        └─ Tokens: 145                                 │
│  │  │                                                       │
│  │  └─ Output: {result: "☑ Successfully added..."}          │
│  │                                                          │
│  └─ Total Tokens: 568 | Total Cost: $0.00012                │
```

```
│                                                                │
└────────────────────────────────────────────────────────────────┘

[Click any node to see full I/O]
[Filter by tags: user:123, transactional]
[Compare with other traces]
```

Analytics Dashboard:

```
┌────────────────────────────────────────────────────────────────┐
│  Project: expense-tracker-ai-langx                             │
│  Time Range: Last 7 days                                       │
├────────────────────────────────────────────────────────────────┤
│                                                                 │
│  📊 PERFORMANCE METRICS                                         │
│  ├─ Latency p50: 2.1s                                          │
│  ├─ Latency p95: 4.8s                                          │
│  ├─ Latency p99: 7.2s                                          │
│  └─ Success Rate: 98.5%                                        │
│                                                                 │
│  💰 COST TRACKING                                               │
│  ├─ Total Cost (7d): $12.45                                    │
│  ├─ Cost per Request: $0.00023 (avg)                           │
│  ├─ By Model:                                                  │
│  │  ├─ gpt-4o-mini: $11.20 (90%)                              │
│  │  └─ text-embedding-3-small: $1.25 (10%)                    │
│  └─ Trend: ↘ -15% vs last week                                 │
│                                                                 │
│  🎯 INTENT DISTRIBUTION                                         │
│  ├─ expense_operation: 65%                                     │
│  ├─ rag_question: 20%                                          │
│  ├─ reconciliation: 10%                                        │
│  └─ general_chat: 5%                                           │
│                                                                 │
│  🚀 TOOL USAGE                                                  │
│  ├─ create_expense: 320 calls                                  │
│  ├─ list_expenses: 180 calls                                   │
│  ├─ modify_expense: 45 calls                                   │
│  ├─ delete_expense: 12 calls                                   │
│  └─ clear_expenses: 3 calls                                    │
│                                                                 │
│  ❌ ERROR ANALYSIS                                              │
│  ├─ Total Errors: 8 (1.5%)                                     │
│  ├─ By Type:                                                   │
│  │  ├─ Validation: 5                                           │
│  │  ├─ Network: 2                                              │
│  │  └─ Timeout: 1                                              │
│  └─ [View Error Details →]                                     │
│                                                                 │
└────────────────────────────────────────────────────────────────┘
```

6.4 Filtering and Search

By Tags:

```
Filter: tags includes "user:123"
→ Shows all traces for user 123

Filter: tags includes "transactional" AND "env:production"
→ Shows production transactional operations only
```

By Metadata:

```
Filter: metadata.sessionId == "session123"
→ Shows all requests in that session

Filter: metadata.feature == "expense_creation"
→ Shows expense creation traces only
```

By Performance:

```
Filter: duration > 5000ms
→ Shows slow requests (>5s)

Filter: cost > 0.001
→ Shows expensive requests
```

By Status:

```
Filter: status == "error"
→ Shows only failed traces

Filter: status == "success" AND duration > 3000ms
→ Shows successful but slow requests
```

6.5 Comparison Mode

Compare Two Traces:

```
Trace A: abc123 (3.5s, $0.00012, success)
Trace B: def456 (7.2s, $0.00034, success)

DIFFERENCES:
├─ Latency: +105% slower
├─ Cost: +183% more expensive
```

```
  ├─ Tokens: 568 vs 1432 (+152%)
  └─ Tool calls: 1 vs 3

ANALYSIS:
- Trace B called list_expenses twice (unnecessary)
- Trace B used more detailed prompts (longer context)

RECOMMENDATION:
- Cache list_expenses results
- Optimize prompt length
```

6.6 A/B Testing

Test Prompt Variations:

```
// Version A: Original prompt
const promptA = PromptTemplate.fromTemplate(`You are an expense assistant...`);

// Version B: More detailed prompt
const promptB = PromptTemplate.fromTemplate(`You are an AI expense assistant with
expertise in...`);

// Tag with version
const llmA = new ChatOpenAI({
  tags: ['transactional', 'prompt:v1']
});

const llmB = new ChatOpenAI({
  tags: ['transactional', 'prompt:v2']
});

// In LangSmith, compare metrics:
// Filter: tags includes "prompt:v1"
// Filter: tags includes "prompt:v2"
// Compare: Latency, cost, success rate, user feedback
```

6.7 Custom Feedback

Collect User Feedback:

```
import { Client } from "langsmith";

const client = new Client({
  apiKey: process.env.LANGCHAIN_API_KEY
});

// After getting response
await client.createFeedback(runId, "thumbs_up", {
  score: 1.0,
```

```
  comment: "Accurate response"
});

// Or thumbs down
await client.createFeedback(runId, "thumbs_down", {
  score: 0.0,
  comment: "Wrong category assigned"
});
```

View Feedback in Dashboard:

```
Traces with thumbs_down:
├─ Run abc123: Wrong category (score: 0.0)
│  └─ Issue: Classified "taxi" as "food" instead of "transport"
│  └─ Action: Update category normalization rules
│
├─ Run def456: Slow response (score: 0.3)
│  └─ Issue: 8.5s latency
│  └─ Action: Add caching for list_expenses
```

---

7. End-to-End Examples

7.1 Example 1: Add Expense

User Request: "Add 500 for lunch today"

Step-by-Step Execution:

```
1. HTTP REQUEST
   POST /ai/chat
   Headers: { Authorization: "Bearer jwt..." }
   Body: {
     "message": "Add 500 for lunch today",
     "history": []
   }

2. EXPRESS MIDDLEWARE
   ├─ authMiddleware: Verify JWT → Extract userId: 123
   ├─ Rate limiter: Check request count → OK
   └─ Body parser: Parse JSON → message, history

3. INTENT ROUTER GRAPH
   ├─ START
   │
   ├─ Node: classifyIntent
   │  ├─ Create LLM (gpt-4o-mini, temp=0)
   │  ├─ Build classification prompt
   │  ├─ Invoke LLM
```

```
    │  │  ├─ Input: "Add 500 for lunch today"
    │  │  └─ Output: {
    │  │       "intent": "expense_operation",
    │  │       "confidence": 0.95,
    │  │       "entities": {
    │  │         "action": "add",
    │  │         "amount": 500,
    │  │         "category": "lunch"
    │  │       }
    │  │     }
    │  └─ Return: { intent: "expense_operation", confidence: 0.95 }
    │
    ├─ Conditional Edge: intent="expense_operation" → handleExpenseOperation
    │
    ├─ Node: handleExpenseOperation
    │  └─ Call: executeExpenseAgent()
    │
    └─ END (return result)

4. AGENT EXECUTOR
    ├─ Create agent
    │  ├─ LLM: gpt-4o-mini (temp=0.7)
    │  ├─ Tools: [create_expense, list_expenses, modify_expense, delete_expense,
clear_expenses]
    │  └─ Prompt: System + Human + Agent Scratchpad
    │
    ├─ Iteration 1: LLM decides what to do
    │  ├─ LLM Call #1
    │  │  ├─ Input:
    │  │  │  System: "You are an expense assistant..."
    │  │  │  Human: "Add 500 for lunch today"
    │  │  │  Tools: [5 tool schemas]
    │  │  │
    │  │  └─ Output:
    │  │     {
    │  │       "tool_calls": [{
    │  │         "name": "create_expense",
    │  │         "arguments": {
    │  │           "amount": 500,
    │  │           "category": "Food",
    │  │           "description": "Lunch",
    │  │           "date": "today"
    │  │         }
    │  │       }]
    │  │     }
    │
    ├─ Execute tool: create_expense
    │  ├─ Validate args (Zod schema)
    │  │  ├─ amount: 500 ☑
    │  │  ├─ category: "Food" ☑
    │  │  ├─ description: "Lunch" ☑
    │  │  └─ date: "today" ☑
    │  │
    │  ├─ Normalize inputs
```

```
    │  │  ├─ category: "Food" → Find category by name
    │  │  │  └─ Backend GET /categories → Find "Food" (id: 1)
    │  │  └─ date: "today" → "2026-02-09"
    │  │
    │  ├─ Call backend
    │  │  ├─ Backend POST /expenses
    │  │  │  Headers: { Authorization: "Bearer jwt..." }
    │  │  │  Body: {
    │  │  │    "amount": 500,
    │  │  │    "category_id": 1,
    │  │  │    "description": "Lunch",
    │  │  │    "date": "2026-02-09"
    │  │  │  }
    │  │  │
    │  │  └─ Response: {
    │  │       "success": true,
    │  │       "expense": {
    │  │         "id": 456,
    │  │         "amount": 500,
    │  │         "category_id": 1,
    │  │         "description": "Lunch",
    │  │         "date": "2026-02-09",
    │  │         "created_at": "2026-02-09T10:30:00Z"
    │  │       }
    │  │     }
    │  │
    │  └─ Return tool result (JSON string):
    │     {
    │       "success": true,
    │       "message": "Expense of ₹500 for Food added successfully on 2026-02-
09.",
    │       "expense": { ... }
    │     }
    │
    ├─ Add tool result to conversation
    │  ├─ AI message: [tool call]
    │  └─ Tool message: [tool result]
    │
    ├─ Iteration 2: LLM sees result and responds
    │  ├─ LLM Call #2
    │  │  ├─ Input:
    │  │  │  System: "You are an expense assistant..."
    │  │  │  Human: "Add 500 for lunch today"
    │  │  │  AI: [tool call: create_expense]
    │  │  │  Tool: [success result]
    │  │  │
    │  │  └─ Output:
    │  │     {
    │  │       "content": "✅ Successfully added ₹500 for Food (Lunch) on 2026-02-
09. Your expense has been saved!"
    │  │     }
    │  │
    │  └─ Agent decides to stop (no more tool calls)
    │
```

```
    └─ Return: {
         "output": "✅ Successfully added ₹500 for Food...",
         "intermediateSteps": [...]
       }

5. RESPONSE TO FRONTEND
   HTTP 200 OK
   {
     "reply": "✅ Successfully added ₹500 for Food (Lunch) on 2026-02-09. Your
expense has been saved!"
   }

6. LANGSMITH TRACE (Automatic)
   Trace ID: abc123-expense-creation
   ├─ Total Duration: 3.5s
   ├─ Total Cost: $0.00012
   ├─ Total Tokens: 568
   │
   ├─ IntentRouterGraph (3.5s)
   │  ├─ classifyIntent (1.2s)
   │  │  └─ ChatOpenAI: 234 tokens, $0.000023
   │  │
   │  └─ handleExpenseOperation (2.3s)
   │     └─ AgentExecutor (2.3s)
   │        ├─ LLM #1: 189 tokens, $0.000019 (0.8s)
   │        ├─ Tool: create_expense (1.2s)
   │        │  └─ Backend API: 0.9s
   │        └─ LLM #2: 145 tokens, $0.000014 (0.3s)
   │
   ├─ Tags: ['expense-tracker', 'transactional', 'user:123']
   └─ Metadata: { traceId, userId, timestamp }
```

7.2 Example 2: Query Expenses

User Request: "Show me my expenses from last week"

Execution:

```
1. INTENT CLASSIFICATION
   Intent: expense_operation (list/query action)
   Confidence: 0.92

2. AGENT EXECUTOR
   ├─ LLM Call #1: Decide to use list_expenses tool
   │  └─ Tool call: list_expenses({ dateFrom: "2026-02-02", dateTo: "2026-02-09"
})
   │
   ├─ Execute tool: list_expenses
   │  ├─ Normalize dates
   │  │  ├─ "last week" → Start: 2026-02-02, End: 2026-02-09
   │  │  └─ Convert to ISO format
```

```
   │  │
   │  ├─ Call backend
   │  │  └─ GET /expenses?dateFrom=2026-02-02&dateTo=2026-02-09
   │  │     Response: {
   │  │       "expenses": [
   │  │         { id: 456, amount: 500, category: "Food", description: "Lunch",
date: "2026-02-09" },
   │  │         { id: 455, amount: 1200, category: "Groceries", description:
"Weekly shopping", date: "2026-02-07" },
   │  │         { id: 454, amount: 300, category: "Transport", description:
"Taxi", date: "2026-02-05" }
   │  │       ],
   │  │       "total": 2000,
   │  │       "count": 3
   │  │     }
   │  │
   │  └─ Return: {"expenses": [...], "total": 2000, "count": 3}
   │
   └─ LLM Call #2: Format response
      └─ Output: "Here are your expenses from last week:

1. **Feb 9, 2026** - ₹500 (Food: Lunch)
2. **Feb 7, 2026** - ₹1,200 (Groceries: Weekly shopping)
3. **Feb 5, 2026** - ₹300 (Transport: Taxi)

**Total**: ₹2,000 (3 expenses)"

3. RESPONSE
   {
     "reply": "Here are your expenses from last week: ..."
   }
```

7.3 Example 3: RAG Retrieval

User Request: "What expenses are listed in the PDF I uploaded?"

Execution:

```
1. INTENT CLASSIFICATION
   Intent: rag_question
   Confidence: 0.97

2. RAG QA HANDLER
   └─ handleRAGQuestion()

3. CHECK DOCUMENTS
   ├─ Query: getUserDocuments(userId: 123)
   └─ Result: Found 45 chunks from 2 PDFs

4. RAG QA CHAIN
   ├─ Create chain
```

```
      │  ├─ Vector store: MemoryVectorStore
      │  ├─ Retriever: asRetriever(k=5, filter=userId:123)
      │  └─ LLM: gpt-4o-mini (temp=0)
      │
      ├─ Embed query
      │  ├─ Input: "What expenses are listed in the PDF I uploaded?"
      │  ├─ Embeddings: OpenAIEmbeddings (text-embedding-3-small)
      │  └─ Vector: [0.123, -0.456, ...] (1536 dimensions)
      │
      ├─ Search vector store
      │  ├─ Cosine similarity with all user chunks
      │  ├─ Filter: metadata.userId === 123
      │  ├─ Sort by score
      │  └─ Top 5 chunks:
      │     1. Chunk 12: "Groceries - ₹1,250 on 15/01/2026" (score: 0.87)
      │     2. Chunk 23: "Restaurant Lunch - ₹450 on 18/01/2026" (score: 0.85)
      │     3. Chunk 34: "Taxi fare - ₹300 on 20/01/2026" (score: 0.82)
      │     4. Chunk 7: "Coffee - ₹120 on 12/01/2026" (score: 0.79)
      │     5. Chunk 19: "Movie tickets - ₹600 on 17/01/2026" (score: 0.76)
      │
      ├─ Format context
      │  └─ Context string:
      │     """
      │     [Source 1 - receipt_jan_2026.pdf]: Groceries - ₹1,250 on 15/01/2026
      │     [Source 2 - receipt_jan_2026.pdf]: Restaurant Lunch - ₹450 on 18/01/2026
      │     [Source 3 - expenses_jan.pdf]: Taxi fare - ₹300 on 20/01/2026
      │     [Source 4 - receipt_jan_2026.pdf]: Coffee - ₹120 on 12/01/2026
      │     [Source 5 - expenses_jan.pdf]: Movie tickets - ₹600 on 17/01/2026
      │     """
      │
      ├─ Build prompt
      │  └─ Prompt:
      │     """
      │     You are an AI assistant answering questions about uploaded PDF documents.
      │
      │     Context:
      │     [Source 1 - receipt_jan_2026.pdf]: Groceries - ₹1,250 on 15/01/2026
      │     ...
      │
      │     Question: What expenses are listed in the PDF I uploaded?
      │
      │     Answer based ONLY on the provided context.
      │     """
      │
      ├─ Invoke LLM
      │  └─ Response:
      │     "Based on your uploaded PDFs, here are the expenses I found:
      │
      │     1. **Groceries** - ₹1,250 (Jan 15, 2026)
      │     2. **Restaurant Lunch** - ₹450 (Jan 18, 2026)
      │     3. **Taxi fare** - ₹300 (Jan 20, 2026)
      │     4. **Coffee** - ₹120 (Jan 12, 2026)
      │     5. **Movie tickets** - ₹600 (Jan 17, 2026)
      │
```

```
   │     **Total from PDFs**: ₹2,720
   │
   │     These expenses appear across two documents: receipt_jan_2026.pdf and
expenses_jan.pdf."
   │
   └─ Return: {
        answer: "Based on your uploaded PDFs...",
        sources: [
          { filename: "receipt_jan_2026.pdf", snippet: "Groceries - ₹1,250..." },
          { filename: "receipt_jan_2026.pdf", snippet: "Restaurant Lunch -
₹450..." },
          ...
        ]
      }

5. RESPONSE
   {
     "reply": "Based on your uploaded PDFs, here are the expenses I found: ..."
   }

6. LANGSMITH TRACE
   ├─ classifyIntent: 0.9s
   ├─ handleRAGQuestion: 3.2s
   │  ├─ getUserDocuments: 0.1s
   │  └─ QA Chain: 3.1s
   │     ├─ Embed query: 0.3s (text-embedding-3-small)
   │     ├─ Vector search: 0.2s (45 chunks, found 5)
   │     └─ LLM generation: 2.6s (gpt-4o-mini)
   │
   └─ Total: 4.1s, $0.00018
```

7.4 Example 4: Reconciliation

User Request: "Compare my PDF expenses with what's in the app"

Execution:

```
1. INTENT CLASSIFICATION
   Intent: rag_compare (Note: NOT reconciliation, just comparison)
   Confidence: 0.93

2. EXTRACT PDF EXPENSES
   ├─ Get all user chunks from vector store
   ├─ Use regex-based expense extractor (deterministic)
   │  ├─ Pattern: ₹?\d+\.?\d* (amount)
   │  ├─ Pattern: \d{2}/\d{2}/\d{4} (date)
   │  └─ Extract context for description
   │
   └─ Result: [
        { amount: 1250, date: "2026-01-15", description: "Groceries" },
        { amount: 450, date: "2026-01-18", description: "Restaurant Lunch" },
```

```
        { amount: 300, date: "2026-01-20", description: "Taxi fare" },
        { amount: 120, date: "2026-01-12", description: "Coffee" },
        { amount: 600, date: "2026-01-17", description: "Movie tickets" }
      ]

3. FETCH APP EXPENSES
   ├─ Backend GET /expenses
   └─ Result: [
        { id: 401, amount: 1250, date: "2026-01-15", category: "Groceries" },
        { id: 402, amount: 300, date: "2026-01-20", category: "Transport" },
        { id: 403, amount: 800, date: "2026-01-22", category: "Food" }
      ]

4. COMPARE (Deterministic Algorithm)
   ├─ For each PDF expense, find best match in app
   │  ├─ Match criteria:
   │  │  ├─ Amount difference <= ₹1
   │  │  ├─ Date exact match
   │  │  └─ Description similarity >= 0.5 (Jaccard)
   │  │
   │  └─ Matching:
   │     ├─ PDF: Groceries ₹1250 (Jan 15)
   │     │  → APP: Groceries ₹1250 (Jan 15) ☑ EXACT MATCH (score: 1.0)
   │     │
   │     ├─ PDF: Restaurant Lunch ₹450 (Jan 18)
   │     │  → APP: No match ❌ MISSING IN APP
   │     │
   │     ├─ PDF: Taxi fare ₹300 (Jan 20)
   │     │  → APP: Transport ₹300 (Jan 20) ☑ MATCH (score: 0.6)
   │     │
   │     ├─ PDF: Coffee ₹120 (Jan 12)
   │     │  → APP: No match ❌ MISSING IN APP
   │     │
   │     └─ PDF: Movie tickets ₹600 (Jan 17)
   │        → APP: No match ❌ MISSING IN APP
   │
   └─ Result:
      {
        matched: [
          { pdf: Groceries ₹1250, app: Groceries ₹1250, matchScore: 1.0 },
          { pdf: Taxi ₹300, app: Transport ₹300, matchScore: 0.6 }
        ],
        pdfOnly: [
          { amount: 450, description: "Restaurant Lunch" },
          { amount: 120, description: "Coffee" },
          { amount: 600, description: "Movie tickets" }
        ],
        appOnly: [
          { id: 403, amount: 800, description: "Food", date: "2026-01-22" }
        ],
        summary: {
          totalMatched: 2,
          totalPdfOnly: 3,
          totalAppOnly: 1
```

```
        }
      }

5. GENERATE EXPLANATION (LLM)
   ├─ Input: Diff result (structured)
   ├─ LLM: Generate natural language explanation
   └─ Output:
      "I've compared your PDF expenses with what's in the app:

      **✅ Matched (2)**
      - Groceries ₹1,250 on Jan 15 (exact match)
      - Taxi fare ₹300 on Jan 20 (matched as Transport)

      **📄 In PDF but NOT in app (3)**
      - Restaurant Lunch ₹450 on Jan 18
      - Coffee ₹120 on Jan 12
      - Movie tickets ₹600 on Jan 17

      **📱 In app but NOT in PDF (1)**
      - Food ₹800 on Jan 22

      **Summary**
      - Total in PDF: ₹2,720 (5 expenses)
      - Total in app: ₹2,350 (3 expenses)
      - Difference: ₹370

      **Recommendation**: Consider adding the 3 missing expenses to your app."

6. RESPONSE
   {
     "reply": "I've compared your PDF expenses with what's in the app: ..."
   }

7. LANGSMITH TRACE
   ├─ classifyIntent: 0.8s
   ├─ handleRagCompare: 4.5s
   │  ├─ extractPdfExpenses: 0.5s (regex-based, deterministic)
   │  ├─ fetchAppExpenses: 0.7s (backend API)
   │  ├─ compareExpenses: 0.2s (deterministic algorithm)
   │  └─ explainDiff: 3.1s (LLM)
   │
   └─ Total: 5.3s, $0.00025
```

---

8. Custom vs Framework Comparison

8.1 Architecture Comparison

Aspect	Custom (ai/)	Framework (ai-langx/)
Orchestration	Manual while loop	AgentExecutor automatic

Aspect	Custom (ai/)	Framework (ai-langx/)
Tool Definition	JSON Schema objects	Zod + StructuredTool classes
Tool Execution	Manual executeTool()	Automatic via agent
RAG Pipeline	Custom implementation	LangChain components
Vector Store	Custom in-memory + JSON	MemoryVectorStore
Similarity Search	Manual cosine similarity	Built-in similaritySearch()
State Management	Manual context passing	LangGraph StateGraph
Workflow Visualization	None (read code)	LangSmith graph view
Observability	Manual console.log	LangSmith automatic
Error Handling	Custom per function	Standardized by framework
Lines of Code	~2000	~800

8.2 Tool-Calling Loop Comparison

Custom Implementation (`ai/src/llm/agent.js`):

```
// MANUAL LOOP
async function processChatMessage(userMessage, authToken, history) {
  const messages = [...history, { role: "user", content: userMessage }];
  let toolIterationCount = 0;
  const MAX_ITERATIONS = 5;

  // Initial LLM call
  let response = await callLLMWithTimeout(messages);
  let responseMessage = response.choices[0].message;

  // MANUAL LOOP: Check for tool calls
  while (responseMessage.tool_calls && responseMessage.tool_calls.length > 0) {
    toolIterationCount++;

    // Safety check
    if (toolIterationCount > MAX_ITERATIONS) {
      return "Error: Too many operations...";
    }

    // Add assistant's message
    messages.push(responseMessage);

    // Execute all tool calls manually
    for (const toolCall of responseMessage.tool_calls) {
      const toolName = toolCall.function.name;
      const toolArgs = JSON.parse(toolCall.function.arguments);

      // Manual tool execution
      const result = await executeTool(toolName, toolArgs, authToken);
```

```
      // Add tool result manually
      messages.push({
        role: "tool",
        tool_call_id: toolCall.id,
        content: JSON.stringify(result)
      });
    }

    // Call LLM again
    response = await callLLMWithTimeout(messages);
    responseMessage = response.choices[0].message;
  }

  return responseMessage.content;
}
```

Framework Implementation (`ai-langx/src/agents/expense.agent.js`):

```
// AUTOMATIC LOOP via AgentExecutor
const agent = await createOpenAIToolsAgent({ llm, tools, prompt });
const executor = new AgentExecutor({
  agent,
  tools,
  maxIterations: 5,
  returnIntermediateSteps: true
});

const result = await executor.invoke({
  input: userMessage,
  chat_history: history
});

// AgentExecutor handles:
// ☑ Tool schema passing
// ☑ Tool call parsing
// ☑ Tool execution
// ☑ Message management
// ☑ Loop until completion
// ☑ Error handling
// ☑ LangSmith tracing
```

Benefits of Framework:

- ☑ 90% less code (10 lines vs 100 lines)
- ☑ No loop bugs (battle-tested)
- ☑ Automatic tracing
- ☑ Consistent behavior
- ☑ Easy to extend

8.3 RAG Pipeline Comparison

Custom Implementation (`ai/src/handlers/ragQaHandler.js`):

```
// MANUAL RAG
export const handleRagQA = async (userMessage, authToken, userId) => {
  // 1. Retrieve chunks manually
  const retrievedChunks = await searchSimilarChunks(userMessage, userId, 5);

  if (retrievedChunks.length === 0) {
    return "No documents found...";
  }

  // 2. Build context manually
  const context = retrievedChunks
    .map((chunk, idx) => `[Source ${idx + 1}]: ${chunk.text}`)
    .join('\n\n');

  // 3. Create prompt manually
  const prompt = `You are an AI assistant...

Document Context:
${context}

User Question: ${userMessage}

Answer:`;

  // 4. Call LLM manually
  const response = await openai.chat.completions.create({
    model: 'gpt-4o-mini',
    messages: [{ role: 'user', content: prompt }],
    temperature: 0.3
  });

  return response.choices[0].message.content;
};
```

Framework Implementation (`ai-langx/src/rag/chains/qa.chain.js`):

```
// AUTOMATIC RAG via RetrievalQAChain
const vectorStore = await getVectorStore();
const retriever = vectorStore.asRetriever({ k: 5 });
const chain = RetrievalQAChain.fromLLM(llm, retriever, {
  prompt: qaPrompt,
  returnSourceDocuments: true
});

const result = await chain.call({ query: userMessage });
```

```
// Chain handles:
// ☑ Embedding query
// ☑ Vector search
// ☑ Context formatting
// ☑ LLM invocation
// ☑ Source attribution
```

Benefits of Framework:

- ☑ 80% less code
- ☑ Source documents included
- ☑ Automatic tracing
- ☑ Easy to swap retrievers

8.4 Observability Comparison

Custom Implementation:

```
// MANUAL LOGGING everywhere
console.log('[LLM Agent] Calling LLM for initial response');
console.log('[LLM Agent] Tool calls:', responseMessage.tool_calls?.length);
console.log('[LLM Agent] Executing tool:', toolName);
console.log('[LLM Agent] Tool result:', result);

// Manual token tracking
const totalTokensUsed = response.usage?.total_tokens || 0;
console.log(`[Cost] ${traceId} | ${userId} | ${model} | ${totalTokensUsed}
tokens`);

// Manual trace ID generation and propagation
const traceId = generateTraceId();
// Pass traceId to every function manually
```

Problems:

- ❌ Logs scattered across files
- ❌ Hard to correlate
- ❌ No visualization
- ❌ Manual cost calculation
- ❌ No performance metrics

Framework Implementation:

```
// ZERO LOGGING CODE
const llm = new ChatOpenAI({
  modelName: "gpt-4o-mini",
  tags: ['expense-tracker', 'transactional', 'user:123'],
  metadata: { traceId, userId }
});
```

```
// LangSmith automatically captures:
// ☑ All LLM calls
// ☑ All tool executions
// ☑ All chain steps
// ☑ Token usage
// ☑ Cost
// ☑ Latency
// ☑ Errors
```

Benefits:

- ☑ Zero code for tracing
- ☑ Visual trace tree in dashboard
- ☑ Automatic cost tracking
- ☑ Performance analytics
- ☑ Error drill-down
- ☑ A/B testing support

8.5 Maintenance Comparison

Task	Custom	Framework
Add new tool	Write JSON schema + run function + register (50 lines)	Extend StructuredTool class (20 lines)
Update OpenAI SDK	Update all direct API calls (10+ files)	Update LangChain dependency (1 file)
Add streaming	Implement SSE manually (100+ lines)	Set `streaming: true` (1 line)
Switch LLM provider	Rewrite all LLM calls	Change LLM class (e.g., ChatOpenAI → ChatAnthropic)
Add memory	Implement conversation storage	Use ConversationBufferMemory
Debug production issue	Parse logs + reconstruct flow (30 min)	Open LangSmith trace (2 min)
Optimize prompts	Deploy new code	Update in LangSmith (no deploy)
Add new handler	Write new handler + routing (100 lines)	Add LangGraph node + edge (30 lines)

9. Demo Script

9.1 Demo Setup

Prerequisites:

1. Backend running on port 3000
2. ai-langx running on port 3002

3. LangSmith tracing enabled
4. Sample PDF uploaded

Environment Variables:

```
# OpenAI
OPENAI_API_KEY=sk-...

# LangSmith
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=ls_...
LANGCHAIN_PROJECT=expense-tracker-demo

# Backend
BACKEND_URL=http://localhost:3000
```

9.2 Demo Flow

Demo 1: Simple Expense Creation

Setup:

```
# Open LangSmith dashboard
open https://smith.langchain.com

# Filter by project
Filter: project == "expense-tracker-demo"
```

Execute:

```
curl -X POST http://localhost:3002/ai/chat \
  -H "Authorization: Bearer $JWT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "message": "Add 500 for lunch today",
    "history": []
  }'
```

Expected Response:

```
{
  "reply": "✅ Successfully added ₹500 for Food on 2026-02-09. Your expense has
been saved!"
}
```

Show in LangSmith:

1. Go to Traces → Find latest trace
2. Expand trace tree
3. Show IntentRouterGraph execution
4. Show classifyIntent node (LLM call)
5. Show handleExpenseOperation node (AgentExecutor)
6. Show tool execution (create_expense)
7. Point out:
    - Total latency: ~3.5s
    - Total cost: ~$0.00012
    - Token breakdown by step
    - Full I/O at each step

Demo 2: Multi-Step Operation

Execute:

```
curl -X POST http://localhost:3002/ai/chat \
  -H "Authorization: Bearer $JWT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "message": "Add 500 for lunch, then show me todays total",
    "history": []
  }'
```

Show in LangSmith:

1. Agent makes 2 tool calls:
    - create_expense
    - list_expenses (with date filter)
2. Point out iteration count: 2
3. Show intermediate steps
4. Show how agent reasoning works

Demo 3: RAG Question Answering

Execute:

```
curl -X POST http://localhost:3002/ai/chat \
  -H "Authorization: Bearer $JWT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "message": "What expenses are in the PDF I uploaded?",
    "history": []
  }'
```

Show in LangSmith:

1. Intent classification: rag_question
2. RAG chain execution:
   - Query embedding
   - Vector search (show retrieved chunks)
   - Context building
   - LLM generation
3. Show source documents returned
4. Compare latency: RAG vs Transactional

Demo 4: Error Handling

Execute:

```
curl -X POST http://localhost:3002/ai/chat \
  -H "Authorization: Bearer $JWT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "message": "Add -500 for lunch",
    "history": []
  }'
```

Show in LangSmith:

1. Tool validation error (Zod catches negative amount)
2. Error message shown in trace
3. Agent recovers and asks user for positive amount
4. Show error classification in LangSmith

Demo 5: Compare with Custom Implementation

Setup:

```
# Terminal 1: Custom implementation (port 3001)
cd ai
npm start

# Terminal 2: Framework implementation (port 3002)
cd ai-langx
npm start
```

Execute Same Request on Both:

```
# Custom
curl -X POST http://localhost:3001/ai/chat \
```

```
  -H "Authorization: Bearer $JWT_TOKEN" \
  -d '{"message": "Add 500 for lunch"}'

# Framework
curl -X POST http://localhost:3002/ai/chat \
  -H "Authorization: Bearer $JWT_TOKEN" \
  -d '{"message": "Add 500 for lunch"}'
```

Compare:

1. **Same Result**: Both return same response format
2. **Latency**: Similar (framework ~10-50ms overhead)
3. **Observability**:
   - Custom: console.log scattered
   - Framework: Visual trace in LangSmith
4. **Code**:
   - Custom: ~2000 lines
   - Framework: ~800 lines
5. **Maintenance**:
   - Custom: Manual updates
   - Framework: Centralized updates

9.3 Demo Talking Points

Why Framework?

- ☑ **5-10x faster development**: Weeks → Days
- ☑ **80% less code**: 2000 → 800 lines
- ☑ **Zero-code observability**: LangSmith automatic
- ☑ **Production patterns**: Battle-tested
- ☑ **Easy maintenance**: Standard patterns
- ☑ **Community support**: 500+ integrations

When Custom?

- ⚠ Unique requirements not supported
- ⚠ Maximum performance critical (every ms)
- ⚠ Want full control over every aspect
- ⚠ Unlimited development time

Recommended Approach:

- ☑ Start with framework (speed)
- ☑ Optimize hotspots if needed
- ☑ Mix: Framework orchestration + Custom deterministic logic
- ☑ Best of both worlds

---

10. Appendix

10.1 Key Files Reference

File	Purpose	Lines
`server.js`	Express setup + LangSmith init	181
`src/graphs/intent-router.graph.js`	Intent classification workflow	724
`src/graphs/reconciliation.graph.js`	Multi-step reconciliation	651
`src/agents/expense.agent.js`	AgentExecutor creation	383
`src/tools/createExpense.tool.js`	Create expense tool	291
`src/rag/chains/qa.chain.js`	RAG Q&A chain	408
`src/config/langsmith.config.js`	LangSmith tracing setup	169
`src/routes/chat.js`	Main chat endpoint	246

10.2 Dependencies

```
{
  "langchain": "^0.1.0",
  "@langchain/openai": "^0.0.19",
  "@langchain/langgraph": "^0.0.8",
  "langsmith": "^0.0.58",
  "zod": "^3.22.4",
  "express": "^4.18.2",
  "axios": "^1.6.2",
  "pdf-parse": "^1.1.1"
}
```

10.3 Environment Variables

```
# OpenAI
OPENAI_API_KEY=sk-...

# LangSmith
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=ls_...
LANGCHAIN_PROJECT=expense-tracker-ai-langx
LANGCHAIN_ENDPOINT=https://api.smith.langchain.com

# Backend
BACKEND_URL=http://localhost:3000

# Server
PORT=3002
NODE_ENV=development

# Security
```

```
JWT_SECRET=...
ALLOWED_ORIGINS=http://localhost:4200

# Safety Limits
MAX_AGENT_ITERATIONS=5
AGENT_TIMEOUT_MS=60000
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=100
```

10.4 Useful Commands

```
# Start server
npm start

# Development mode (watch)
npm run dev

# Run tests
npm test

# Check LangSmith traces
open https://smith.langchain.com

# Test endpoint
curl -X POST http://localhost:3002/ai/chat \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"message": "Add 500 for lunch"}'
```

10.5 Learning Resources

- **LangChain Docs**: https://js.langchain.com/docs
- **LangGraph Docs**: https://langchain-ai.github.io/langgraphjs/
- **LangSmith Docs**: https://docs.smith.langchain.com
- **Custom Implementation**: See `ai/` folder for comparison
- **Architecture Plan**: See `notes/AI_LANGX_ARCHITECTURE_PLAN.md`
- **Framework Mapping**: See `notes/AI_FRAMEWORK_MAPPING.md`

End of Deep Dive Document

Document Status: ☑ Complete
Last Updated: February 9, 2026
Maintainer: AI Engineering Team